

NLU course project - Part 1A and Part 1B

Emanuele Ricca (258437)

University of Trento

emanuele.ricca@studenti.unitn.it

1. Introduction

1.1. Part 1.A – GPT2 from Scratch

This section presents the incremental optimization of a custom GPT2-style language model [5] trained from scratch on the Penn TreeBank dataset [3]. Starting from a constrained baseline, the network capacity was expanded by adjusting hidden dimensions, attention heads, layer depth, and feed-forward dimensions. To regularize the scaled configurations, four independent dropout mechanisms [6] were evaluated alongside weight-tying techniques [4]. Each modification was isolated to minimize validation perplexity under the mandatory threshold ($\text{PPL} < 250$). Generative AI utilities assisted with script debugging and formatting.

1.2. Part 1.B – LoRA Fine-Tuning

This section evaluates Parameter-Efficient Fine-Tuning (PEFT) via a manual implementation of Low-Rank Adaptation (LoRA) [1] on a pre-trained GPT2 backbone [5]. Trainable low-rank adapter bottlenecks were integrated directly into the attention projection pathways while core network weights remained completely frozen. The design goal was to satisfy the performance constraint ($\text{PPL} < 250$) while outperforming the custom scratch-built architectures optimized in Part 1.A. AI tools supported scripting logic verification and text layout formatting.

2. Implementation Details

2.1. Part 1.A – GPT2 from Scratch

The core architecture consists of token embeddings, absolute learnable position embeddings, masked causal self-attention [7], and a linear language modeling head. The dimension footprint was modified from a baseline ($d_{\text{model}} = 20, n_{\text{heads}} = 1, num_{\text{layers}} = 1, ff_{\text{dim}} = 20$) up to an expanded capacity configuration ($d_{\text{model}} = 64, n_{\text{heads}} = 2, num_{\text{layers}} = 2, ff_{\text{dim}} = 128$).

Four independent dropout layers [6] were mapped into specific data paths: embedding dropout, attention dropout, projection dropout, and feed-forward dropout. Weight tying [4] was integrated to share exact parameters between the input embedding matrix and the final linear output layer. Optimization was handled via AdamW [2] over 20 epochs with a 3-epoch linear warmup, an early stopping patience of 3, and an adjusted learning rate scaled down from 0.01 to 0.005 for larger sizes to stabilize convergence.

2.2. Part 1.B – LoRA Fine-Tuning

LoRA adapters [1] were developed manually from scratch without external PEFT libraries. Low-rank decomposition paths were explicitly mapped to modify the linear query, key, and value projection layers inside each self-attention block [7]. For a frozen base weight matrix $W_0 \in \mathbb{R}^{d \times k}$, the adapted forward

pass is computed as:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} B A$$

where $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$ represent the trainable low-rank matrices, $r = 4$ is the inner rank bottleneck, and $\alpha \in \{8, 16\}$ is the constant scaling multiplier. All original base parameters were strictly frozen. Training utilized AdamW [2], a tuned learning rate of 0.0005, and a 3-epoch linear warmup.

3. Results and Evaluation

3.1. Part 1.A – GPT2 from Scratch

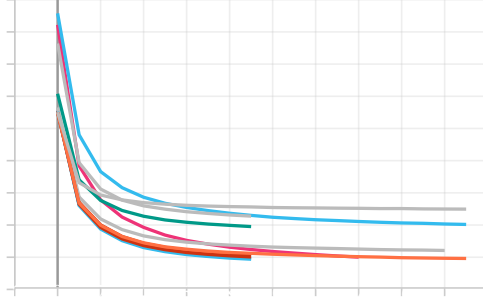
Performance was measured using cross-entropy validation loss converted to Perplexity ($\text{PPL} = \exp(\text{loss})$). Table 1 outlines the performance metrics gathered across the incremental optimization sequence.

Table 1: Incremental optimization perplexities for Part 1.A.

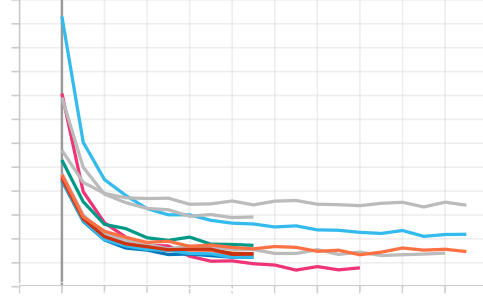
Incremental Modification Variant	Dev PPL	Test PPL
Baseline Layout ($lr = 0.01$)	56.68	49.51
+ Bigger d_{model} (32, $ff_{\text{dim}} = 64$)	46.68	41.51
+ More Layers ($num_{\text{layers}} = 2$)	46.75	41.45
+ More Heads ($n_{\text{heads}} = 2$)	46.18	40.77
+ Embedding Dropout (0.2)	48.65	43.02
+ Embedding Dropout (0.05)	46.52	41.25
+ Attention Dropout (0.05)*	46.56	N/A
+ Feed-Forward Dropout (0.1)	46.17	40.96
+ Projection Dropout (0.05)	46.91	41.57
+ Weight Tying (WT, $lr = 0.01$)	54.45	48.10
+ WT + FF Drop 0.1 ($lr = 0.005$)	50.53	44.77
+ WT + FF Drop 0.1 (Scaled Up)	43.47	38.81

Empirical metrics show that expanding d_{model} from 20 to 32 delivered the primary boost in modeling capabilities. The model did not exhibit signs of prominent overfitting, which rendered embedding and projection dropouts [6] slightly detrimental; only feed-forward dropout at 0.1 provided isolated regularization benefits. Weight tying [4] initially degraded perplexity due to representation constraints on a tight parameter budget, but drastically lowered parameter counts, allowing us to safely scale up the network dimensions ($d_{\text{model}} = 64, ff_{\text{dim}} = 128$). This final configuration yielded the lowest test PPL of 38.81.

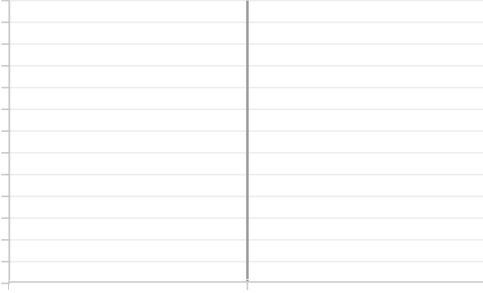
TensorBoard logs display three clear trajectory archetypes: the baseline curve shows slow convergence; scaling d_{model} shifts the entire curve downward; introducing weight tying initially shifts curves upward before the scaled-up model establishes an optimized low-loss trajectory. The curves are explicitly locked in place within Figure 1.



(a) Training Loss



(b) Development PPL



(c) Test PPL Points

Figure 1: *TensorBoard* convergence curves and evaluation metrics for Part 1.A (SVG format, locked single column layout).

3.2. Part 1.B – LoRA Fine-Tuning

The fine-tuned configurations evaluated on the Penn TreeBank partitions are compiled in Table 2, restricted to the current section column.

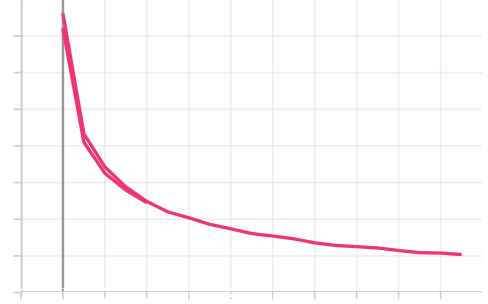
Table 2: *LoRA* fine-tuning parameters and perplexity metrics.

Configuration	Rank	Alpha	Dev PPL	Test PPL
LoRA Run 1*	4	16	24.29	N/A
LoRA Run 2 (Best)	4	8	23.24	20.99

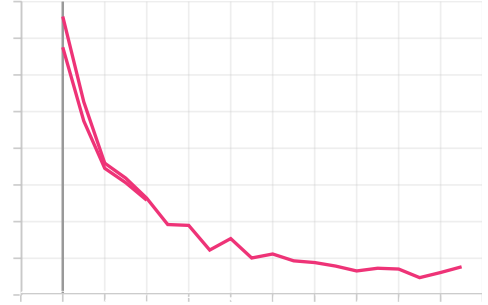
The manual LoRA framework converged exceptionally fast, plateauing fully within 3 to 4 epochs. Leveraging a large pre-trained representation base [5] allowed the adapter matrices to completely outperform the from-scratch models from Part 1.A, easily satisfying target requirements with a final test perplexity of 20.99 [1].

TensorBoard tracks highlight a unique flat convergence profile. While scratch models started with high initial perplexities

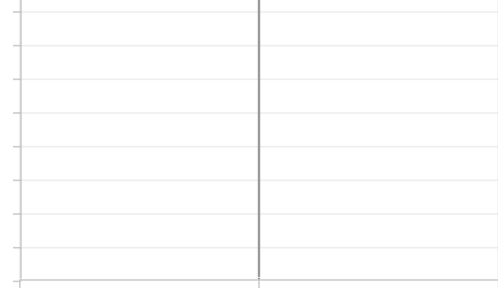
and required extended training to reach the 40s, LoRA launched immediately at Epoch 1 with a dev PPL near 26, concluding at a validation PPL of 23.24. Although each individual step is wall-clock slower due to computing gradients through the frozen backbone, the rapid epoch convergence minimizes total computational budgets. The run with $\alpha = 16$ was stopped early as its development metrics did not warrant test set translation. The trajectories are strictly anchored below in Figure 2.



(a) Training Loss Tracking



(b) Flat Dev PPL Profile



(c) Comparative Test PPL

Figure 2: *TensorBoard* trajectories demonstrating the flat optimization profile of LoRA fine-tuning (SVG format, locked single column layout).

4. References

- [1] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *ICLR*, 2022.
- [2] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [3] Mitchell P Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.

- [4] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.
- [5] Alec Radford, Jeffrey Wu, Rewon Child, Dario Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
- [6] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Luan Kaiser, and Illia Polosukhin. Attention is all you need. *NeurIPS*, 2017.